BUS TICKET BOOKING SYSTEM

C++ Object-Oriented Programming Project Report

Project Code	CPP15
Enrollment Number	IU2541230425
Project Title	Bus Ticket Booking System
Implementation	C++17, console-based application
Data Storage	Text files (pipe-delimited records)

Academic project demonstrating object-oriented programming, record management, ticket booking, cancellation, and file persistence.

Prepared for academic project submission

1. Abstract

The Bus Ticket Booking System is a console-based C++ application that demonstrates how a small transport service can maintain bus records and manage passenger ticket bookings. The program supports adding, viewing, searching, updating and deleting bus records; booking and cancelling tickets; calculating fares from sample route distances; generating a summary report; and saving data to text files so it can be loaded again in a later session.

2. Problem Statement

Maintaining bus details, available seats and passenger bookings manually can lead to inconsistent records and make searching difficult. This project provides a structured, menu-driven application that organizes these tasks using C++ classes and file handling. It is an academic prototype and is not connected to a live transport provider or payment service.

3. Objectives

- Maintain bus details including bus number, name, seat capacity and route.
- Search and update bus records and prevent unsafe deletion where active bookings exist.
- Record passenger information and issue a ticket for a selected bus and route.
- Cancel an active ticket and return its seat to availability.
- Calculate a sample fare based on route distance.
- Store and reload bus and ticket information using text files.
- Demonstrate encapsulation, inheritance, polymorphism, constructors/destructors, dynamic memory and file handling.

4. Scope and Features

- Bus management: add, display, search, update and delete records.
- Ticket management: book tickets, view ticket details, cancel bookings.
- Reporting: display a summary of bus and ticket records.
- Persistence: Data/buses.txt and Data/tickets.txt store records between runs.
- Sample data: the application initializes demonstration routes and buses when no bus data is saved.

5. System Requirements

Software: Windows 10/11, a C++17 compiler such as GCC/MinGW-w64, and optionally Visual Studio Code as the editor. **Hardware:** a general-purpose computer capable of running Windows and a C++ compiler. No internet connection is required after the compiler and editor are installed.

6. System Design and Class Responsibilities

Class	Responsibility
Person	Abstract base class for common passenger identity; declares a virtual detail-display function.
Passenger	Inherits from Person and stores passenger name, age and phone number.
Route	Stores route ID, source, destination and distance; calculates fare.
Bus	Stores bus details, total/available seats and route; reserves/releases seats.
Ticket	Stores ticket ID, bus/route, passenger, seat, fare and active/cancelled state.
FileManager	Loads and saves bus and ticket records in the Data folder.
BusSystem	Coordinates menus, validation, records, booking/cancellation, reporting and memory management.

7. Object-Oriented Programming Concepts

- Classes and objects: real-world entities such as buses, routes, passengers and tickets are represented as classes.
- Encapsulation: data is held in class members and accessed through public methods.
- Inheritance: Passenger derives from Person.
- Runtime polymorphism: Person declares a virtual method that Passenger overrides.
- Constructors and destructors: initialize objects and release resources.
- Dynamic memory: BusSystem manages dynamically allocated arrays and releases them in its destructor.
- File handling: FileManager reads and writes structured records.
- Exception handling: conversion/parsing errors are caught where appropriate.

8. Program Workflow

- Create a BusSystem object and initialize routes.
- Load existing bus and ticket records from the Data folder.
- If no bus records are available, initialize sample bus records.
- Display the main menu and accept the user's selection.
- Validate inputs and execute the selected operation.
- For booking, check the selected bus and available seats, collect passenger details, calculate the fare, create a ticket, reduce availability and save data.
- For cancellation, mark the ticket cancelled, release the seat and save data.
- On exit, save current records and release dynamically allocated resources.

9. Data Storage

The application stores bus data in Data/buses.txt and ticket data in Data/tickets.txt. Each record uses the pipe character (|) between fields. The application should be launched with the project root as its working directory so the relative Data/ path resolves correctly. Do not include the pipe character in names because it is used as a field separator.

10. UML Diagrams

The accompanying Diagrams folder contains the Use Case Diagram, Class Diagram and two Sequence Diagrams. UseCase.svg describes the main actors and actions. ClassDiagram.svg summarizes the system classes and relationships. Sequence1.svg illustrates ticket booking, while Sequence2.svg illustrates bus search. Open the SVG files in a web browser to view them at full size; they can be inserted into a word-processing report if required by the instructor.

11. Test Plan

Test	Expected result
Display bus records	Show the available sample/saved bus records.
Search existing bus (e.g., 101)	Display matching bus information.
Search unknown bus	Show a not-found message without changing data.
Book ticket with valid passenger details	Create a ticket ID and reduce available seats by one.
Cancel active ticket	Mark ticket cancelled and release one seat.
Delete bus with active booking	Reject deletion or otherwise protect active ticket consistency.
Persistence	After exit and restart, previously saved records remain available.
Invalid age/phone input	Reject invalid values and request valid input.

Testing status: These are proposed test cases and expected outcomes, not a claim that every test has been performed on the student's computer. Run the application and record actual results before final submission.

12. Limitations and Future Enhancements

This prototype is console-based, uses local text files, and uses sample routes/fares. It does not provide live bus inventory, concurrent booking control, online payments, authentication, or a graphical interface. Possible future enhancements include a database, graphical user interface, seat map, user/admin login, stronger validation, and integration with a real transport service.

13. How to Build and Run

On Windows with g++ installed, open the project root folder and run build_and_run.bat. Alternatively, open a terminal at the project root and compile all .cpp files in Source using the command provided in README.txt. The executable is created in Executable/ and should be launched from the project root to ensure that Data/ is found.

14. Final Submission Checklist

- Compile and run the project on the target computer.
- Capture genuine screenshots of the menu, bus list, successful booking, cancellation and report, if screenshots are required.
- Record actual test outcomes and replace any placeholders required by the institution.
- Verify institution-specific cover-page, declaration, acknowledgement, index and signature requirements against the official template.

Prepared as a general academic project report. The exact institution-specific template requirements should be checked against the official assignment documents.